



GUÍA DE ACTIVIDAD PRÁCTICA

Ingeniería para el Desarrollo Móvil · Unidad 2 · Una feature de punta a punta, en capas

Programa	Facultad de Ingeniería · Posgrado	Asignatura	Ingeniería para el Desarrollo Móvil
Unidad	Unidad 2 · Práctica opcional	Unidad / Corte	2 · Corte 1
Modalidad	Individual o en pareja	Periodo	2026-B
Tipo	Formativa (opcional, sin nota)	Entrega	Aula Moodle, según indique el tutor

Objetivos

- Documentar una feature con la plantilla «feature» del marco de gobierno móvil antes de programarla.
- Implementar el dominio de la feature en TypeScript puro y probarlo sin Angular.
- Construir la capa de datos en dos pasos —repositorio falso en memoria y repositorio HTTP con DTO y mapper— intercambiables con una sola línea.
- Conectar una pantalla de lista y un formulario reactivo a un ViewModel con los estados cargando, cargado, vacío, error y offline.

1. Objetivo

Construir **una** feature completa, en capas, y comprobar en código las decisiones que el H2 te pide argumentar. La práctica es **opcional y no calificable**; prepara directamente la **Actividad de aprendizaje 2 (20 %, individual)**, que vence el **domingo 25 de octubre a las 11:59 p. m.** Dedícale entre 3 y 4 horas.



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL – SNIES 2828

Elige la feature: una de **tu proyecto ABP** (recomendado: la que tenga lista y formulario) o la feature **visitas de Bitácora de Campo**, la app de referencia del curso. El orden es de adentro hacia afuera: documento → dominio → datos → presentación.

Nota: Todo lo que produzcas es reutilizable: la Parte A alimenta lo que tu publicación del foro dice sobre estructura y estado (sección 01 del marco); las Partes B y C, lo que dice sobre APIs, tokens y datos (sección 03); la Parte D, lo que dice sobre interfaz (sección 02).

2. Parte A — Documenta la feature con la plantilla del marco

Copia la plantilla «feature» de [Plantillas del proyecto](<https://code-corhuila.github.io/ova-web/2026-B/ingenieria-desarrollo-movil/abp/plantillas/>) y llénala **antes** de escribir código (método SDD: diseño primero).

Apartado de la plantilla	Qué escribir	Ejemplo con Bitácora · visitas
Propósito	Una o dos frases sobre el problema del usuario	«El técnico ve sus visitas aun sin señal y registra una nueva»
Pantallas	Ruta, pantalla y propósito de cada una	/visitas (lista) y el modal «Nueva visita»
Estado	Patrón, objeto de estado y transiciones	ViewModel con EstadoListaVisitas: cargando → cargado, vacío, error u offline
Datos	Casos de uso, repositorio, endpoints, persistencia	ListarVisitas, RegistrarVisita; GET y POST /visitas; caché de lectura
Bordes y estados	La lista de verificación de la plantilla	Cargando, vacío, error, offline; permiso denegado no aplica
Decisiones	Los ADR de los que depende	ADR-001 patrón de estado; ADR-002 persistencia local
Pruebas	Qué se prueba y en qué nivel	Caso de uso y ViewModel con el repositorio falso

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 – 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 – 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Para el endpoint principal llena también la plantilla «**integración de endpoint**» (mapeo DTO → dominio, tabla de errores y estrategia de caché).

Atención: Si no puedes llenar «Estado» y «Bordes y estados», todavía no estás listo para la Parte D. Ese vacío es exactamente lo que después aparece como una pantalla en blanco.

3. Parte B — Dominio: entidad, caso de uso y contrato

1. Crea `features/<tu-feature>/domain/` y define la entidad con campos `readonly`; esa carpeta no importa nada de `@angular`, `@ionic` ni `@capacitor` (búscalos para verificarlo).
2. Declara el contrato del repositorio como **clase abstracta** (sirve como token de inyección).
3. Escribe un caso de uso por acción de negocio: como mínimo, listar y registrar.
4. Si la feature tiene una regla de negocio, escríbela como función pura (como «una visita cancelada exige motivo»).
5. Prueba el caso de uso **sin Angular**, con el repositorio falso de la Parte C.

```
// tab: Dominio
export interface Visita { readonly id: string; readonly cliente: string; readonly fecha:
Date; readonly estado: EstadoVisita; }
export interface Listado<T> { readonly datos: readonly T[]; readonly desactualizado:
boolean; }

export abstract class VisitasRepository {
  abstract listar(): Observable<Listado<Visita>>;
  abstract registrar(borrador: BorradorVisita): Observable<Visita>;
}

export class ListarVisitas {
  constructor(private readonly repo: VisitasRepository) {}
  ejecutar(): Observable<Listado<Visita>> { /* tu regla: ordenar, filtrar, agrupar... */ }
}
```



```
// tab: Prueba sin Angular
// Solo usa describe, it y expect: corre igual con Jasmine/Karma o con Vitest.
describe('ListarVisitas', () => {
  it('entrega primero la visita más reciente', async () => {
    const repo = new VisitasEnMemoriaRepository([visita('a', '2026-10-01'), visita('b', '2026-10-09')]);
    const listado = await firstValueFrom(new ListarVisitas(repo).ejecutar());
    expect(listado.datos.map(v => v.id)).toEqual(['b', 'a']);
  });
});
```

4. Parte C — Datos: primero falso, después HTTP

Paso 1 · Repositorio falso. Implementa el contrato en memoria, con latencia simulada y un modo que te permita forzar cada estado de la pantalla sin depender de la red:

```
export class VisitasEnMemoriaRepository extends VisitasRepository {
  modo: 'normal' | 'vacio' | 'sin-red' | 'error' = 'normal';
  constructor(private datos: Visita[] = VISITAS_DE_EJEMPLO) { super(); }

  listar(): Observable<Listado<Visita>> {
    return timer(400).pipe(switchMap(() => { // latencia simulada
      if (this.modo === 'sin-red') return throwError(() => new ErrorDeDatos('sin-
conexion'));
      if (this.modo === 'error') return throwError(() => new ErrorDeDatos('transitorio'));
      return of({ datos: this.modo === 'vacio' ? [] : this.datos, desactualizado:
false });
    }));
  }

  registrar(b: BorradorVisita): Observable<Visita> {
    const nueva: Visita = { ...aVisitaNueva(b), id: crypto.randomUUID() };
    this.datos = [nueva, ...this.datos];
    return timer(400).pipe(map(() => nueva));
  }
}
```



Paso 2 · Repositorio HTTP. Escribe el DTO como espejo exacto del JSON de tu API (o de la API simulada local de la Bitácora: POST /auth/login y CRUD de /visitas), el mapper DTO → entidad que **rechaza** datos malformados con un error de formato (pruébalo con una fecha inválida), y el repositorio que usa el HttpClient de core/ y traduce cada falla con aErrorDeDatos.

Paso 3 · El cambio de una línea. Si las capas están bien trazadas, pasar del falso al real es cambiar, en la raíz de composición, useClass: VisitasEnMemoriaRepository por useClass: VisitasHttpRepository. Nada más.

Si tu feature lo pide, envuelve el repositorio HTTP con un **decorador de caché de lectura** que devuelva **desactualizado: true** cuando sirva la copia local (es un mínimo del ABP).

Consejo: Si al cambiar de repositorio tuviste que tocar la página o el ViewModel, hay una fuga entre capas. Encuéntrala y anótala: es un excelente ejemplo para tu publicación del foro.

5. Parte D — Presentación: ViewModel, lista y formulario

1. Crea el ViewModel de la lista, provisto en la página, con **un** signal estado (unión discriminada) e intenciones **recargar()** y **registrar()**.
2. En la plantilla, cubre los cinco estados con **@if ... @else if; ion-refresher** solo emite la intención de recargar, y toda acción por gesto (deslizar, presión larga) existe también como botón.
3. El formulario reactivo vive en un modal: **NonNullableFormBuilder**, al menos tres validadores de formato y un validador de grupo que **consulta** la regla del dominio; devuelve el borrador con **dismiss()**.
4. El ViewModel registra con **exhaustMap** (dobles toques) y recarga la lista al terminar.
5. Ningún color ni espacio escrito a mano: solo tokens del tema.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Fuerza cada estado con el modo del repositorio falso y toma una captura de cada uno:

Estado	Cómo forzarlo	Qué debe verse
Cargando	Sube la latencia simulada a 3 s	Esqueleto de la lista, no un spinner eterno
Cargado	<code>modo = 'normal'</code>	Lista con el estado legible en texto, no solo en color
Vacío	<code>modo = 'vacio'</code>	Mensaje y acción «Registrar la primera visita»
Offline	<code>modo = 'sin-red'</code>	Aviso de sin conexión y «Reintentar»
Error	<code>modo = 'error'</code>	Mensaje y «Reintentar» que funciona dos veces seguidas

Atención: La prueba de «Reintentar dos veces seguidas» no es un capricho: si el segundo intento no hace nada, el `catchError` está fuera del `switchMap` y el flujo murió con el primer error.

6. Entrega y autoevaluación

- **Producto:** PDF de 3 a 5 páginas (U2-practica-<apellido>.pdf; en pareja, los dos apellidos) con el documento de la feature, el código clave o el enlace al repositorio, las cinco capturas y un párrafo final: «Qué cambiaría en mi ADR de estado después de esta práctica».
- **Dónde:** donde indique el tutor en el aula Moodle. Al ser práctica opcional **no tiene nota:** su valor es que el H2 se escriba sobre decisiones probadas.
- [] El documento de la feature tiene llenos «Estado», «Datos» y «Bordes y estados», y se escribió antes del código.
- [] `domain/` no importa nada de Angular, Ionic ni Capacitor.
- [] Hay al menos una prueba del caso de uso con el repositorio falso y una del mapper.
- [] Pasar del repositorio falso al HTTP fue un cambio de una sola línea, y el DTO no aparece fuera de `data/`.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



- [] La pantalla muestra los cinco estados, con una captura de cada uno.
- [] «Reintentar» funciona dos veces seguidas.
- [] El formulario reutiliza la regla del dominio y no la duplica.
- [] Ningún color ni tamaño está escrito a mano en las pantallas.

Criterio	Logrado	En proceso	Insuficiente
Documento de la feature	Plantilla completa y coherente con el código	Completa, pero el código se apartó	Ausente o escrita después del código
Dominio	Puro y con pruebas	Puro, sin pruebas	Importa infraestructura
Datos	Falso y HTTP intercambiables; DTO y mapper que valida	HTTP sin mapper o sin mapeo de errores	La página usa HttpClient
Presentación	Un ViewModel, cinco estados y el formulario consulta al dominio	Faltan estados o la regla está duplicada	Lógica de negocio en la plantilla

Consejo: Hacia el H2. Cierra con tres frases para tu publicación del foro: la estructura por capas, tu patrón de estado (con la alternativa descartada) y cómo tratas errores, tokens y falta de red. [Guía del proyecto ABP](<https://code-corhuila.github.io/ova-web/2026-B/ingenieria-desarrollo-movil/abp/guia/>).

Nota: Versión imprimible. [Descarga esta práctica en PDF](pdf/Practica-Unidad02.pdf), con el membrete institucional.